基于Docker容器化的软件自动化测试框架研究

佘凤，顾敏
（黄冈职业技术大学，湖北 黄冈 438002）

摘　要：研究围绕Docker容器化技术与主从架构模式，系统设计了一种面向分布式场景的软件自动化测试框架。首先，从整体层面设计了基于容器化的Master-Slave测试体系结构。其次，针对Master节点提出了一种融合负载感知与状态反馈的任务调度与监控联合策略。再次，在此基础上，进一步构建了具备执行效能建模与状态自反馈能力的Slave容器化节点架构。最后，在统一的实验环境下对提出的架构与方法进行了对比测试。实验结果表明，该框架在测试执行效率、资源利用均衡性及测试稳定性等方面均优于传统固定调度策略，具有有效性。

关键词：Docker容器化；自动化测试；主从架构；任务调度；负载感知
DOI:10.16184/j.cnki.comprg.2026.09.002

1　概述

近年来，Docker容器化技术[1,2]因其轻量级隔离、快速部署与良好的可移植性特征，而被逐步引入软件测试领域。相关研究多集中于容器化测试环境的构建与单节点任务的调度[3,4]，但在多节点协同、动态资源调度及测试控制机制方面仍存在架构耦合度高、扩展能力不足等问题。针对上述不足，研究基于Docker容器化技术与主从架构模式[5]，设计了一种软件自动化测试框架的总体架构，并围绕其中的关键技术展开系统分析，通过实验测试对所提方法的有效性与可行性进行了验证，为构建高效、可扩展的软件自动化测试系统提供了一种新的实现思路。

2　总体架构设计

Docker容器化技术是一种基于操作系统级虚拟化的轻量级应用封装与运行机制，其核心思想在于通过容器对应用程序及其依赖环境进行统一封装，并借助Linux内核的命名空间与控制组等机制实现资源隔离与精细化管控，从而在保证运行环境一致性的同时显著降低系统开销。主从架构则是一种典型的分布式控制模式，其中，主节点负责全局任务调度、状态管理与控制决策，而从节点则专注于具体任务的执行与结果反馈。该模式通过明确的职责划分有效提升了系统的可扩展性与整体协调效率，在分布式计算与自动化测试场景中具有较高的工程适用价值。

在此基础上，研究提出了一种面向自动化测试场景的增强型容器化主从测试架构，总体架构设计方案如图1所示。该架构在传统Master-Slave模式之上引入统一资源管理层与事件驱动调度机制，使Master节点不仅承担着测试任务分发功能，还能基于实时反馈信息对Slave容器的生命周期与资源配额进行动态调控，从而实现测试资源的自适应调度与负载均衡。与此同时，各Slave节点以独立容器形式运行测试用例，并通过消息通信机制向Master节点回传执行状态与结果信息。

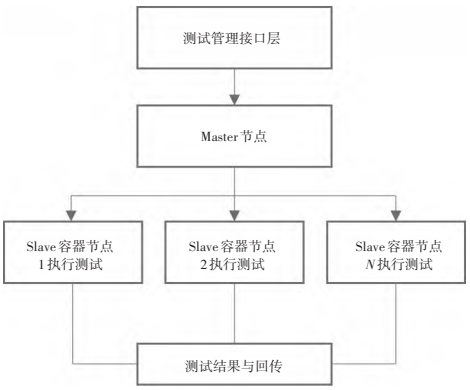


图1　总体架构设计方案

该架构通过将容器化节点的弹性扩展与集中式调度控制相结合，在保证测试环境一致性的同时，提升了自动化测试系统在复杂工程场景下的执行效率与运行稳定性。

3　关键技术研究

3.1　Master节点设计

由图1可知，Master节点中的任务调度与状态监控不仅属于核心功能模块，而且共同构成了系统全局控制与资源优化的决策中枢。其中，任务调度模块决定了测试任务在各Slave容器间的分配策略，直接影响到系统

基金项目：黄冈职业技术学院 工程技术/一般项目“基于虚拟化技术在软件测试中的应用与优化研究”（2023C2012106）。

作者简介：佘凤（1976—），女，本科，副教授，研究方向为软件技术；顾敏（1978—），女，本科，副教授，研究方向为信息技术。

中国知网 https://www.cnki.net

吞吐能力与执行时延，而状态监控模块则通过持续感知各节点的运行状态与资源占用情况，为调度决策提供实时、可量化的反馈依据。

在此基础上，研究提出了一种基于“负载感知—反馈调节”的创新性Master节点任务调度与状态监控联合策略，其核心思想在于将Slave节点的多维运行状态映射为统一的负载评价指标，并据此动态调整任务分配权重。设系统中第i个Slave容器节点在调度时刻t的综合负载水平为$L_i(t)$，则该指标可如公式（1）所示：

$$L_i(t) = \alpha C_i(t) + \beta M_i(t) + \gamma Q_i(t) \quad (1)$$

其中，$C_i(t)$为节点在时刻t的CPU利用率；$M_i(t)$为内存占用率；$Q_i(t)$为当前待执行测试任务队列长度；α、β、β为用于平衡不同资源影响程度的权重系数，用以反映各类资源在测试执行过程中的相对重要性。基于该负载模型，Master节点在进行任务分配时，可针对每个Slave节点计算其调度优先级权重$W_i(t)$，如公式（2）所示：

$$W_i(t) = \frac{1}{L_i(t) + \varepsilon} \quad (2)$$

其中，ε为防止分母为零而引入的微小常数，该权重的物理含义在于：节点负载越低，其获得新测试任务的概率越高。与此同时，状态监控模块以固定或事件触发的方式对各节点状态进行采样，并将最新状态反馈至调度模块，从而形成“监控—评估—调度—再监控”的闭环控制过程，实现测试资源的自适应调节与动态负载均衡。上述联合策略在Master节点中的策略设计如图2所示。

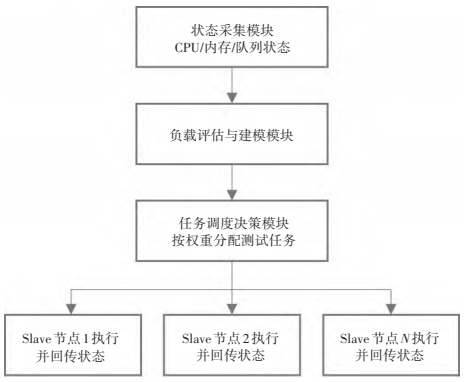


图2　Master节点中的策略设计

该策略能够在测试负载动态变化的情况下维持系统整体运行的稳定性与高效性，为容器化自动化测试框架在大规模、多任务场景中的应用提供了可行的理论支撑与工程实现路径。

3.2　Slave容器化节点设计

在上述基于负载感知与反馈调节的Master节点联合策略约束下，Slave容器化节点不再仅作为被动的测试任务执行单元，而应演化为具备“状态自描述—执行自调节—结果可量化反馈”能力的自治执行节点，从而在体系结构层面与Master节点形成信息对称、决策协同的闭环关系。基于这一设计目标，可构建一种面向自动化测试场景的增强型Slave容器化节点架构，其核心思想在于通过对测试执行过程进行细粒度建模，使节点运行状态与任务执行能力能够被形式化表达并实时反馈。

具体而言，设第i个Slave节点在时刻t接收的测试任务集合为$T_i(t) = \{T_{i,1}, T_{i,2}, \cdots, T_{i,k}\}$，则该节点的瞬时执行能力可定义为一个执行效能函数$E_i(t)$，如公式（3）所示：

$$E_i(t) = \frac{1}{k} \sum_{j=1}^{k} \frac{1}{\tau_{ij}(t)} \quad (3)$$

其中，$\tau_{ij}(t)$为任务T_{ij}在节点i上的实际执行时长，该指标用于刻画节点在当前资源条件下对测试任务的处理效率。在执行过程中，Slave节点通过内部资源监测模块持续采集CPU利用率$C_i(t)$、内存占用率$M_i(t)$及I/O等候时间$I_i(t)$，并据此构建节点的局部运行状态向量，如公式（4）所示：

$$S_i(t) = [C_i(t), M_i(t), I_i(t), E_i(t)] \quad (4)$$

该状态向量不仅反映了节点的资源占用情况，还刻画了其测试执行性能特征。基于$S_i(t)$，Slave节点可对当前任务队列进行自适应调节，例如，通过限制并发执行度或调整任务执行顺序，使节点负载保持在可控区间内；与此同时，该状态向量作为结构化反馈信息被周期性或事件触发式地上报至Master节点，为其全局调度权重$W_i(t)$的更新提供高可信度输入。

从体系结构角度看，该Slave节点在逻辑上由测试执行层、资源感知层与状态反馈层构成，Slave节点的内部工作机制如图3所示。

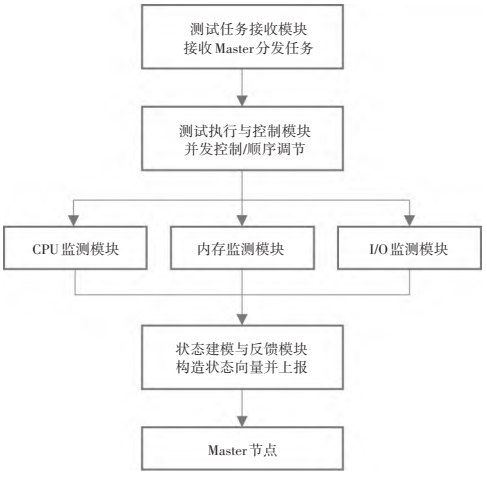


图3　Slave节点的内部工作机制

中国知网　https://www.cnki.net

通过在Slave容器化节点内部引入以执行效能建模与状态向量反馈为核心的结构设计，该架构实现了在节点层面对测试负载变化的主动感知与局部调节，并在系统层面与Master节点的联合调度策略形成协同优化机制。

4 实验与分析

4.1 实验环境搭建

为验证提出的基于Docker容器化的Master-Slave自动化测试架构在真实场景下的可行性与有效性，需要构建一个具备代表性的实验开发与运行环境，以确保测试结果具有工程参考价值。研究中用于系统开发、部署与实验验证的核心开发与实验环境配置，如表1所示。

表1 核心开发与实验环境配置

环境类别	具体配置
操作系统	Ubuntu Server 20.04 LTS
容器平台	Docker Engine 24.x
容器编排	Docker Compose
开发语言	Python 3.9
通信机制	gRPC / RESTful API
测试框架	PyTest

实验以Linux服务器操作系统为基础，通过Docker提供统一、可隔离的测试运行环境，能够有效消除不同节点间的软件依赖差异。Python语言及其生态为Master节点调度逻辑与Slave节点执行控制的实现提供了支持，而基于gRPC或REST的通信机制则保障了主从节点之间状态信息与控制指令的高效传输。

基于表1，实验方案通过在同一物理服务器上部署1个Master节点与8个Slave容器化节点来构建完整的自动化测试架构，其中Master节点以独立容器形式运行，负责测试任务调度、状态监控与结果汇总，而每个Slave节点均采用统一镜像启动，以保证测试环境的一致性。在核心参数设置方面，状态采集周期设定为5 s，CPU、内存与任务队列长度在负载模型中的权重系数分别取0.5、0.3、0.2，调度权重计算中的平滑因子设为0.001，同时限制单个Slave节点的最大并发测试任务数为4，以避免资源争用导致的性能退化。

4.2 性能对比测试

在测试阶段，选取包含200个测试用例的自动化测试集，采用批量任务注入方式持续向Master节点提交测试请求，对系统总执行时间、节点平均负载及测试成功率等指标进行对比分析。在相同测试规模条件下，不同调度策略对应的示例实验结果数据，如表2所示。其中，对比对象为固定轮询调度策略与提出的负载感知联合调度策略。

表2 不同调度策略下的实验结果

策略	总执行时间（s）	平均CPU利用率（%）	平均内存利用率（%）	测试成功率（%）
固定轮询调度	1 282	62.4	58.7	95.6
负载感知联合调度	980	78.1	71.3	99.3

从总体执行效率的角度来看，在相同测试用例规模下，联合调度策略对应的总执行时间明显低于固定调度策略，这表明通过在Master节点中引入基于负载评估的动态任务分配机制，能够有效减少节点空闲与资源等待现象，从系统层面提升测试任务的并行执行效率。进一步观察资源利用情况可以发现，联合调度策略下各Slave节点的平均CPU利用率与内存利用率均保持在较高且相对稳定的区间，说明该方法在充分挖掘节点计算能力的同时，并未产生显著的资源波动或过载问题，这从侧面验证了负载建模与反馈调节机制在资源分配上的合理性。同时，在测试结果可靠性方面，联合调度策略下的测试成功率呈现更加理想的数值水平。这一现象表明，通过限制节点并发度并结合状态感知进行任务调度，可以有效降低因资源竞争而导致的测试失败概率，从而提升整体测试过程的稳定性与结果可信度。

5 结语

研究围绕基于Docker容器化的软件自动化测试问题，从体系结构设计、关键机制建模与实验验证等多个层面展开了系统研究，构建了一种以Master节点集中调度与Slave节点自治执行相结合的自动化测试框架。实验结果表明，所提架构不仅能够在保证测试环境一致性的前提下提升系统的整体执行效率，还在资源利用率与测试结果稳定性方面表现出较好的综合性能，具备较强的实际应用价值。

参考文献

[1] 刘思尧, 李强, 李斌. 基于Docker技术的容器隔离性研究[J]. 软件, 2015, 36(4): 110-113.

[2] 孙伟. Docker容器的性能监控和日志服务的设计与实现[J]. 网络安全和信息化, 2025(12): 164-166.

[3] 李耀, 周亚明, 罗辉, 等. 基于Docker的嵌入式软件测试研究[J]. 电子技术与软件工程, 2020(12): 67-69.

[4] 王逸璠. 软件测试多维评估系统的设计与实现[D]. 南京: 南京大学, 2020.

[5] 王艳. 基于Docker容器的MySQL数据主从同步技术分析与实践[J]. 信息记录材料, 2024, 25(2): 208-210.

中国知网 https://www.cnki.net